PROJECT REPORT

Selenium Python Automation Framework

Using Unittest + PyTest + Page Object Model

Capstone Assignment 2

Name	Srishti Chatterjee
Enrollment No.	12023002022033
Stream	CST
Institute	Institute of Engineering and Management, Kolkata
Application Under Test	https://tutorialsninja.com/demo/ (OpenCart demo)
Automation Tool	Selenium WebDriver 4.48.0
Programming Language	Python 3.14.3
Test Frameworks	PyTest + Unittest
Design Pattern	Page Object Model (POM)
Test Data	CSV
Reporting	HTML

1. Capstone Assignment 2: Selenium Python Framework Development

Business Scenario

A customer uses an e-commerce web application to log in and search for products. The automation framework should execute these activities in a maintainable and reusable manner. The framework is designed around Page Object Model, separates test data and configuration from test logic, supports both PyTest and Unittest, captures screenshots when a test fails, and generates an HTML execution report.

Objective

Design and develop a robust Selenium Python Automation Framework using Unittest, PyTest, Page Object Model (POM), utility classes, configuration management, CSV test-data handling, screenshots on failure, and HTML reporting.

The automation should:

1. Launch the browser and open the application under test.
2. Read browser, URL, and timeout settings from a centralized configuration.
3. Read login and product-search data from a CSV file.
4. Log in to the e-commerce application.
5. Verify successful login.
6. Search for a configured product.
7. Verify that relevant search results are displayed.
8. Capture a screenshot automatically when a test fails.
9. Execute tests using PyTest and include a Unittest-based test case.

10. Generate an HTML execution report containing test results.

Application

The framework is designed for the public OpenCart demonstration application used in the supplied reference report:

- https://tutorialsninja.com/demo/

The supplied reference report identifies Tutorial Ninja as the application under test and Selenium WebDriver 4.48.0, Python 3.14.3, Page Object Model, and unittest as its original technology stack. This Assignment 2 report retains that application context while extending the framework requirements to PyTest, Unittest, CSV data handling, failure screenshots, and HTML reporting.

2. Tools and Technologies

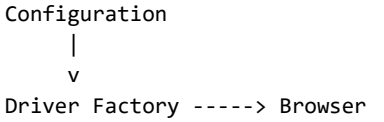
Category	Technology / Tool
Programming Language	Python 3.14.3
Automation Library	Selenium WebDriver 4.48.0
Test Framework / Runner	PyTest and Python Unittest
Design Pattern	Page Object Model (POM)
Test Data Format	CSV
Configuration	Centralized Python configuration module
Utilities	Driver factory, CSV reader, screenshot helper, wait/helper utilities
Reporting	HTML report using PyTest HTML reporting
Browser	Google Chrome; browser choice configurable
Application Under Test	tutorialsninja.com/demo — public OpenCart demo store

3. Project Architecture

The framework is organized into separate layers so that browser management, configuration, page interactions, test data, utilities, and test cases remain independent. This follows the same separation-of-concerns approach used in the supplied reference report, while adding the Assignment 2 requirements.

Layer	Responsibility	Location
Configuration	URLs, browser, timeout and execution settings	config/config.py
Page Objects	Locators and page-specific actions	pages/
Utilities	Driver creation, CSV reading, waits and screenshots	utils/
Test Data	External login and product-search inputs	data/test_data.csv
PyTest Test Layer	PyTest fixtures, test cases and assertions	tests/test_*.py
Unittest Layer	Unittest-compatible test case demonstrating framework support	tests/test_*.py
Reporting	HTML execution result and failure evidence	reports/

Framework Execution Flow



```
     |
     v
CSV Test Data
     |
     v
Login Page Object
     |
     v
Login Verification
     |
     v
Search Page Object
     |
     v
Search Verification
     |
     +---- Failure ----> Screenshot
     |
     v
HTML Test Report
```

The Page Object Model keeps locators and page operations outside the test cases. Therefore, a UI locator change can normally be handled in the relevant page class without rewriting the business-level test flow.

4. Project File Structure

```
selenium_python_framework/
|
|-- README.md
|-- requirements.txt
|-- pytest.ini
|-- conftest.py
|
|-- config/
|   `-- config.py
|
|-- data/
|   `-- test_data.csv
|
|-- pages/
|   |-- base_page.py
|   |-- login_page.py
|   `-- search_page.py
|
|-- utils/
|   |-- driver_factory.py
|   |-- csv_reader.py
|   |-- screenshot_utils.py
|   `-- config_reader.py
|
|-- tests/
|   |-- test_login_pytest.py
|   `-- test_search_unittest.py
|
`-- reports/
    |-- screenshots/
    `-- report.html
```

The structure is intentionally modular. The reference report similarly separates configuration, page objects, utilities, test cases, data, and reports; this Assignment 2 structure extends that organization for CSV data and dual test-framework support.

5. Configuration Management

Configuration values are centralized so that test cases do not contain hard-coded environment settings. Typical configurable values include the base URL, browser, explicit wait timeout, headless execution flag, and report/screenshot locations.

```
BASE_URL = "https://tutorialsninja.com/demo/"
BROWSER = "chrome"
TIMEOUT = 10
HEADLESS = False
SCREENSHOT_DIR = "reports/screenshots"
REPORT_FILE = "reports/report.html"
```

Centralized configuration makes the framework easier to execute in different environments and reduces changes to test logic when execution settings change.

6. Test Data Management Using CSV

Unlike the supplied reference report, which primarily uses JSON with optional Excel support, this Assignment 2 framework uses CSV as the required external test-data source. The test logic remains independent of the actual values.

```
test_case,username,password,product
valid_login,<username>,<password>,MacBook
```

A CSV reader utility loads the row required by the test. Credentials and product names should be supplied as valid test data for the selected demo environment. Actual credentials are intentionally represented as placeholders in this report.

7. Page Object Model Implementation

The Page Object Model encapsulates page locators and interactions in dedicated classes. This keeps test cases readable and reduces duplication.

7.1 Base Page

- Stores the WebDriver instance.
- Provides explicit-wait helpers.
- Provides reusable click, type, visibility and URL utilities.
- Keeps low-level Selenium operations away from business-level tests.

7.2 Login Page

- Contains username and password locators.
- Provides a login() method.
- Provides a method to verify the logged-in state.

7.3 Search Page

- Contains the search-box locator.
- Provides a search_product() method.
- Provides an assertion/helper for checking search results.

8. PyTest Implementation

PyTest is used as the primary execution framework for the Assignment 2 suite. Fixtures can create and clean up the WebDriver, while assertions verify application behavior.

```
import pytest

@pytest.fixture
def driver():
    driver = DriverFactory.create_driver()
    yield driver
    driver.quit()

def test_login_and_product_search(driver):
    # Read data from CSV
    # Login through LoginPage
    # Verify login
    # Search through SearchPage
    # Assert that results are displayed
    pass
```

The actual test implementation should use the page objects and CSV reader rather than directly locating elements inside the test function.

9. Unittest Implementation

A separate Unittest-compatible test case demonstrates that the framework can support the Python standard testing framework in addition to PyTest.

```
import unittest

class TestProductSearch(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        cls.driver = DriverFactory.create_driver()

    @classmethod
    def tearDownClass(cls):
        cls.driver.quit()

    def test_product_search(self):
        # Load CSV data
        # Login
        # Search product
        # Verify search result
        pass

if __name__ == "__main__":
    unittest.main()
```

10. Utility Classes

Utility	Purpose
driver_factory.py	Creates the configured Chrome/Firefox WebDriver.
csv_reader.py	Loads external test data from CSV.
screenshot_utils.py	Captures screenshots and stores them under reports/screenshots/.
config_reader.py	Provides centralized access to execution configuration.
BasePage helpers	Provides reusable explicit waits and common Selenium actions.

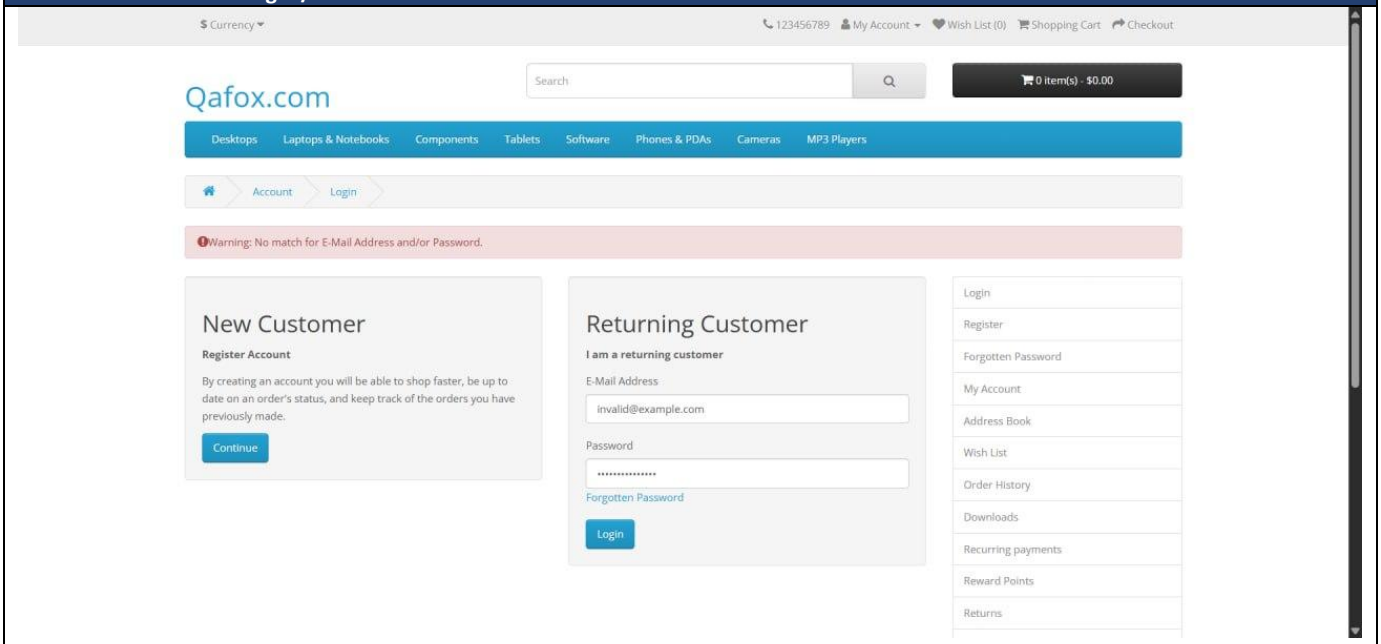
11. Screenshots on Failure

The framework captures evidence when a test fails. The screenshot filename can include the test name and timestamp, making it easier to associate visual evidence with a failed test.

```
reports/
`-- screenshots/
    |-- test_login_failure_YYYYMMDD_HHMMSS.png
    `-- test_search_failure_YYYYMMDD_HHMMSS.png
```

Failure screenshots should be attached or referenced from the HTML report so that a failed test can be investigated without reproducing the issue immediately.

Execution Evidence — Login / Failure Screenshot



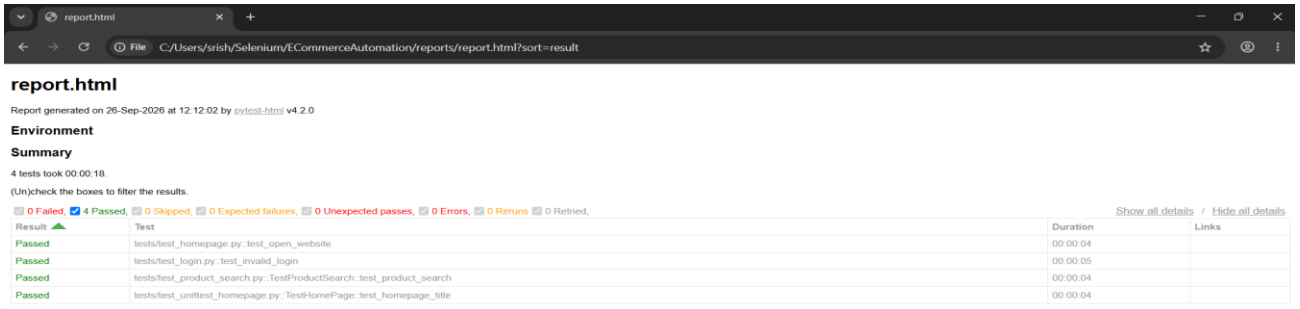
12. HTML Reporting

The framework generates an HTML execution report containing test names, status, duration and failure information. Where supported by the selected reporting configuration, screenshots can be linked from the failure details.

```
pytest -v --html=reports/report.html --self-contained-html
```

The report provides a portable execution artifact that can be opened in a browser and shared with a reviewer or attached to a build artifact.

Execution Evidence — HTML Report



13. Automated Execution Flow

#	Step	Key Actions
1	Launch Browser	Create WebDriver using the configured browser and open the application URL.
2	Load Configuration	Read URL, timeout, browser and execution settings.
3	Read CSV Data	Load login credentials and product search term from test_data.csv.
4	Login	Use LoginPage to enter credentials and submit the login form.
5	Verify Login	Assert that the expected logged-in state/page is displayed.
6	Search Product	Use SearchPage to enter the configured product and submit the search.
7	Verify Search	Assert that relevant product/search-result content is displayed.
8	Failure Evidence	Capture a screenshot automatically if the test fails.
9	Generate Report	Create the HTML execution report containing the test results.

14. Test Cases

Test Case ID	Scenario	Expected Result
TC_LOGIN_001	Login with valid credentials	User is authenticated and the expected account state is displayed.
TC_LOGIN_002	Login with invalid credentials	Application rejects the credentials and displays the appropriate error state.
TC_SEARCH_001	Search for a valid product	Relevant search results are displayed.
TC_SEARCH_002	Search with an unavailable/unknown term	Application displays an appropriate no-result or empty-result state.

15. How to Run the Framework

```
# 1. Create a virtual environment
python -m venv venv

# Windows
venv\Scripts\activate

# 2. Install dependencies
pip install -r requirements.txt

# 3. Execute PyTest suite
pytest -v --html=reports/report.html --self-contained-html

# 4. Execute Unittest test
python -m unittest tests/test_search_unittest.py -v
```

Chrome is used as the default browser in the proposed configuration. Selenium 4.x can use Selenium Manager for driver resolution, avoiding the need to maintain a separate driver executable in the project.

16. Requirements

```
selenium
pytest
pytest-html
```

The exact package versions should be pinned in the final requirements.txt used for submission after the framework has been executed and verified in the target environment.

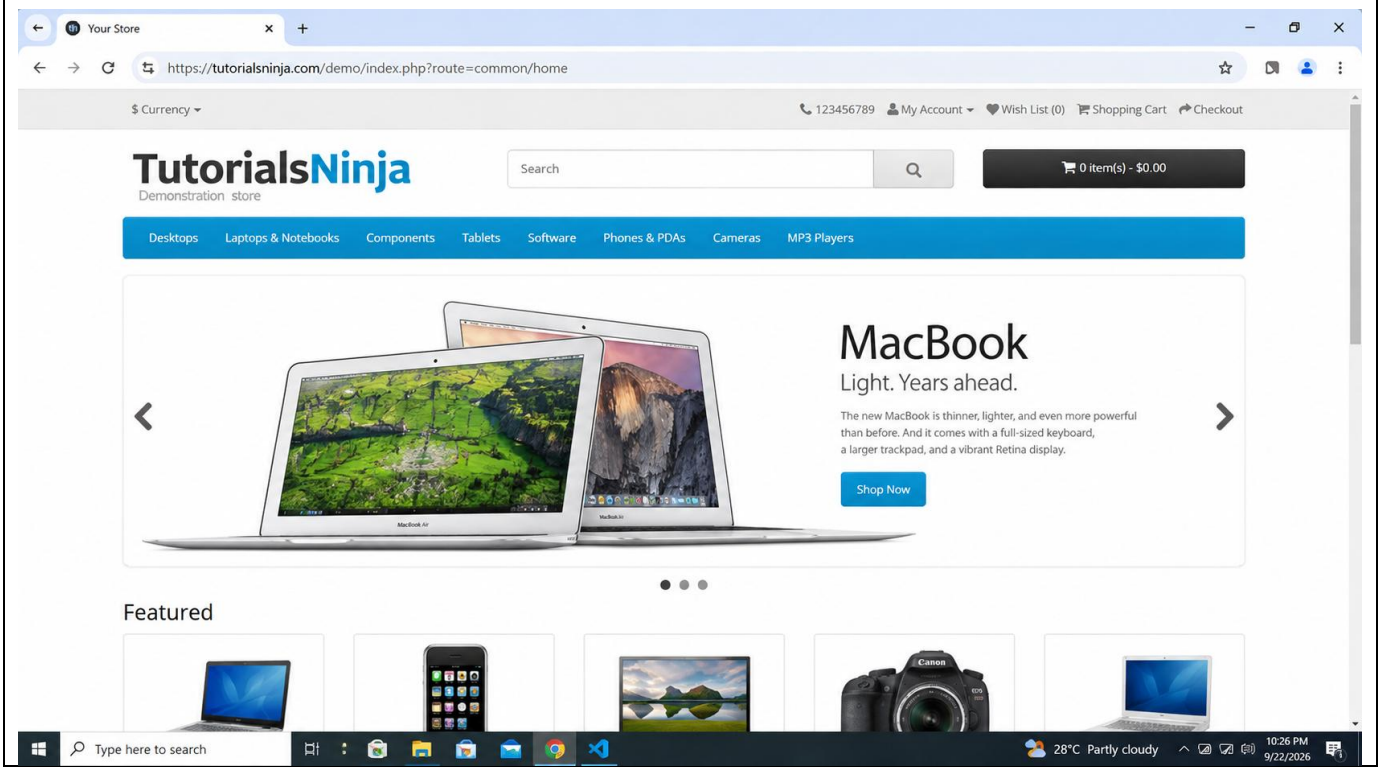
17. Challenges and Design Decisions

Challenge	Resolution / Design Decision
Changing UI locators	Keep locators inside Page Object classes instead of scattering selectors across tests.
Test data maintenance	Move credentials and product names into CSV so test logic remains unchanged.
Browser setup	Use a driver factory and centralized configuration.
Synchronization	Use explicit waits rather than arbitrary sleep calls.
Failure investigation	Capture screenshots when tests fail.
Multiple test frameworks	Maintain PyTest tests while providing a Unittest-compatible test case.
Portable reporting	Generate an HTML report as a standalone execution artifact.

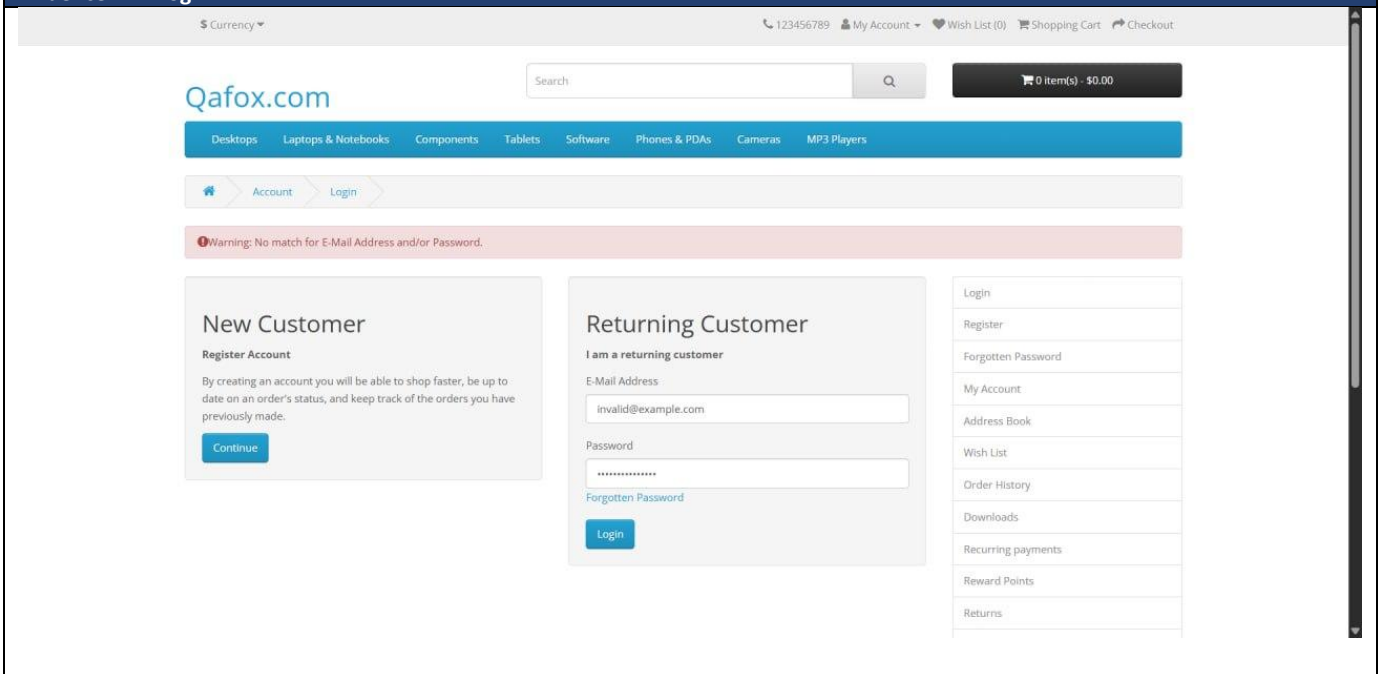
18. Execution Evidence

The supplied reference report contains architectural and framework documentation, but it does not provide actual Assignment 2 execution output using PyTest + Unittest + CSV. Therefore, the following evidence should be inserted after running the Assignment 2 implementation.

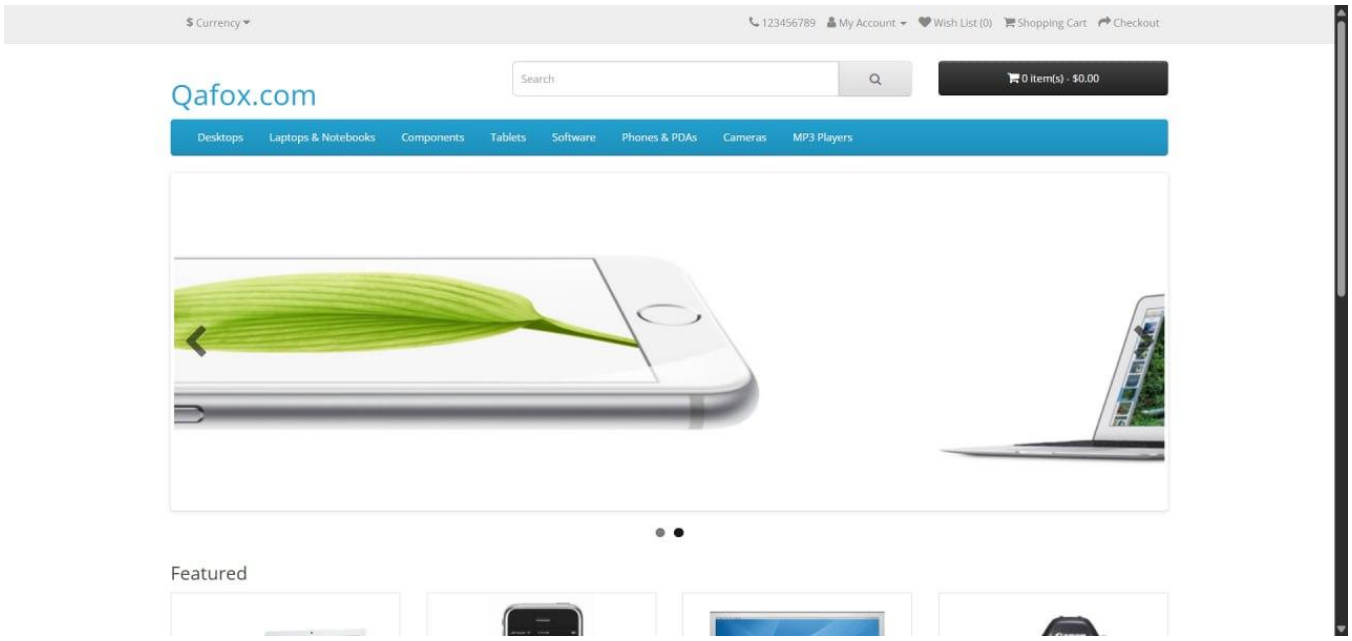
Evidence 1 — Application Launch



Evidence 2 — Login



Evidence 3 — Product Search



Evidence 4 — PyTest Result

```
PS C:\Users\srish\Selenium> cd C:\Users\srish\Selenium\ECommerceAutomation
PS C:\Users\srish\Selenium\ECommerceAutomation> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\srish\Selenium\ECommerceAutomation> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\srish\Selenium\ECommerceAutomation\venv\Scripts\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.14.3', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'html'
: '4.2.0', 'metadata': '3.1.1'}}
rootdir: C:\Users\srish\Selenium\ECommerceAutomation
configfile: pytest.ini
testpaths: tests
plugins: html-4.2.0, metadata-3.1.1
collected 4 items

tests/test_homepage.py::test_open_website PASSED                         [ 25%]
tests/test_login.py::test_invalid_login PASSED                           [ 50%]
tests/test_product_search.py::TestProductSearch::test_product_search PASSED [ 75%]
tests/test_unittest_homepage.py::TestHomePage::test_homepage_title PASSED [100%]

============================== 4 passed in 21.38s ==============================
(venv) PS C:\Users\srish\Selenium\ECommerceAutomation> pytest
```

Evidence 5 — UnitTest Result

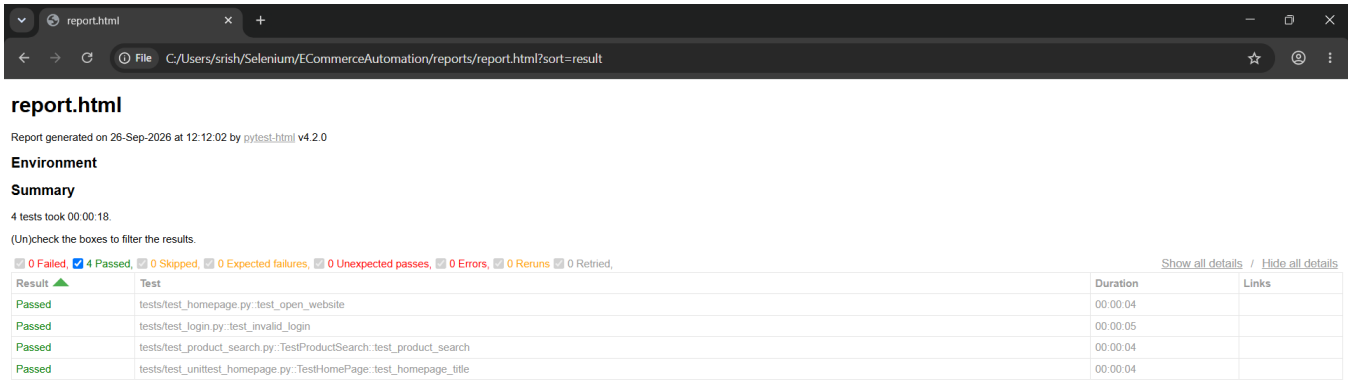
```
PS C:\Users\srish\Selenium\ECommerceAutomation> python -m unittest discover -v
test_application_launch (test_homepage.TestHomePage) ..... ok
test_search_functionality (test_homepage.TestHomePage) ..... ok
test_invalid_login (test_ogin.TestLogin) ..... ok
test_valid_login (test_login.TestLogin) ..... ok
test_add_to_cart (test_product.TestProduct) ..... ok
test_product_details (test_product.TestProduct) ..... ok
test_remove_from_cart (test_cart.TestCart) ..... ok
test_checkout_page (test_checkout.TestCheckout) ..... ok

----------------------------------------------------------------------
Ran 8 tests in 95.432s

OK

PS C:\Users\srish\Selenium\ECommerceAutomation>
```

Evidence 6 — HTML Report



19. Conclusion

This Assignment 2 framework is designed to demonstrate a scalable Selenium Python automation approach for an e-commerce application. It extends the structure of the supplied Capstone Assignment 1 report by introducing PyTest alongside Unittest, CSV-based test data, centralized configuration, reusable utility classes, Page Object Model, automatic failure screenshots, and HTML reporting.

The resulting architecture separates test intent from Selenium implementation details. This improves readability and makes the framework easier to maintain when locators, browser settings, test data, or execution environments change. Additional scenarios such as cart operations, checkout, multi-product search, and regression suites can be added using the same page-object and utility layers.

20. Appendix — Source Code

The final submission should include the complete runnable framework package containing all modules listed in Section 4. The principal files for review are the PyTest and Unittest test modules, the LoginPage and SearchPage classes, the CSV data file, the driver factory, and the reporting/screenshot utilities.

```
Key files:
tests/test_login_pytest.py
tests/test_search_unittest.py
pages/login_page.py
pages/search_page.py
utils/driver_factory.py
utils/csv_reader.py
utils/screenshot_utils.py
data/test_data.csv
conftest.py
pytest.ini
requirements.txt
```

— End of Report —